



14 DE NOVIEMBRE DE 2024

REPORTE DE PRACTICA 5 GNS3

ADMINISTRACIÓN DE SERVICIOS EN RED

ESCUELA SUPERIOR DE CÓMPUTO

RODRIGO JOZAFAT GUTIERREZ MARTINEZ

7CM1



Configuración automática de SSH

Información de la actividad

En esta práctica se implementó una topología como la de la siguiente imagen, en donde se configuraron todas las interfaces de todos los routers, cuidando configurar correctamente las IP's administrativas y usando un enrutamiento dinámico OSPF

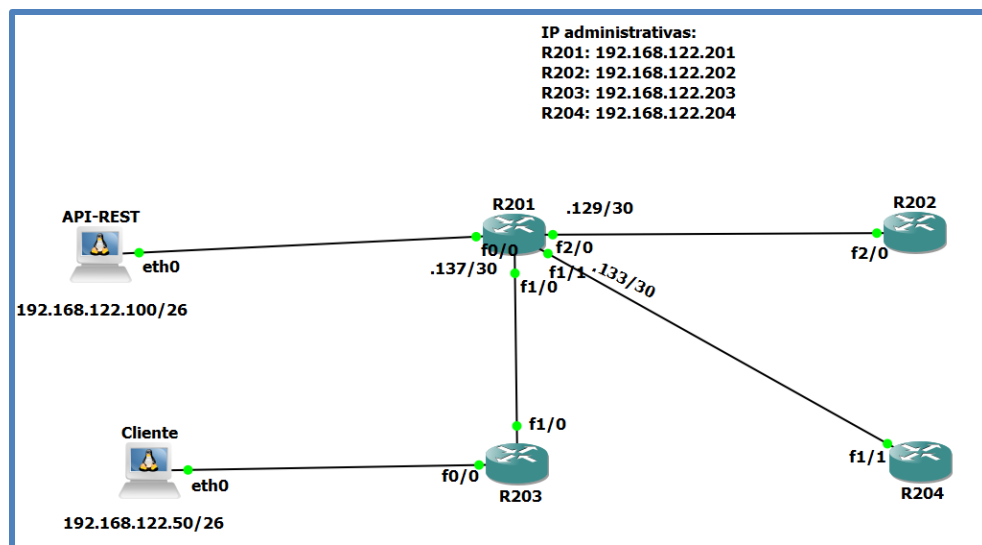


Imagen 1. Topología de la red

En cada dispositivo se especifica la IP utilizada, siendo el Gateway el host más pequeño de cada red, el objetivo de la práctica fue levantar una configuración via Telnet en cada router y crear un usuario y administrador pero sin utilizar SSH, para cada router se veía de la siguiente manera.

```

line con 0
  exec-timeout 0 0
  privilege level 15
  logging synchronous
  stopbits 1
line aux 0
  exec-timeout 0 0
  privilege level 15
  logging synchronous
  stopbits 1
line vty 0 4
  login local
  transport input telnet
!
!
end

```

Imagen 2. Configuración de telnet en cada router

```

hostname R201
!
boot-start-marker
boot-end-marker
!
enable secret 5 $1$G829$AoXA7Bgj8zb3PNDEtioZl0
enable password 7 0055415550
!
no aaa new-model
no ip icmp rate-limit unreachable
!
!
ip cef
no ip domain lookup
!

```

Imagen 3. Usuario y contraseña habilitados en cada router para conectarse via telnet

Se utilizó el siguiente script de Python para conectarse vía Telnet a cada router y hacer el levantamiento de SSH en cada uno de ellos, para esto, se utilizó una API-REST con las librerías pexpect y Flask, las cuales corrieron en el servidor API-REST y cuando se conectaba un usuario y hacia una petición de tipo GET al servidor, se iniciaba la configuración, el comando para conectarse desde el cliente es:

```

curl -X POST http://192.168.122.100:5000/ssh -H "Content-Type: application/json" -d '{"username": "sshuser", "password": "sshpass"}'

```

```

root@API-REST:~# cat script5.py
from flask import Flask, jsonify, request
import pexpect
import time
from typing import List, Dict

app = Flask(__name__)

class RouterConfig:
    def __init__(self, ip: str, username: str = "admin", password: str = "admin"):
        self.ip = ip
        self.username = username
        self.password = password

    def configure_ssh(self, new_username: str, new_password: str) -> Dict[str, str]:
        """Configura SSH en un router mediante Telnet usando pexpect."""
        try:
            print(f"[INFO] Conectando a {self.ip} mediante Telnet...")
            child = pexpect.spawn(f'telnet {self.ip}')
            child.expect('Username:')
            child.sendline(self.username)
            child.expect('Password:')
            child.sendline(self.password)
            child.expect('#')
            time.sleep(1)

            # Entrar en modo configuración
            print(f"[INFO] Configurando SSH en {self.ip}...")
            child.sendline('configure terminal')
            child.expect('#')
            time.sleep(1)

            # Lista de comandos para configurar SSH
            commands = [
                'ip domain-name local.domain',
                'ip ssh rsa keypair-name sshkey',
                'crypto key generate rsa usage-keys label sshkey modulus 2048',
                f'username {new_username} privilege 15 secret {new_password}',
                'line vty 0 4',
                'transport input ssh telnet', # Permitimos telnet también
                'login local',
                'exit',
                'ip ssh version 2',
                'ip ssh authentication-retries 3',
                'ip ssh time-out 60',
                'end',
            ]

```

```

        for cmd in commands:
            print(f"[INFO] Ejecutando comando en {self.ip}: {cmd}")
            child.sendline(cmd)
            # Espera adicional para la generación de claves RSA
            if 'crypto key generate' in cmd:
                child.expect('#', timeout=30)
            else:
                child.expect('#', timeout=5)
            time.sleep(1) # Retraso entre cada comando

        # Cerrar conexión
        child.sendline('exit')
        child.close()
        print(f"[SUCCESS] SSH configurado exitosamente en {self.ip}")

        return {"status": "success", "message": f"SSH configured successfully on {self.ip}"}

    except pexpect.ExceptionPexpect as e:
        error_message = f"Error configuring SSH on {self.ip}: {str(e)}"
        print(f"[ERROR] {error_message}")
        return {"status": "error", "message": error_message}
    except Exception as e:
        error_message = f"Unexpected error on {self.ip}: {str(e)}"
        print(f"[ERROR] {error_message}")
        return {"status": "error", "message": error_message}

@app.route('/ssh', methods=['POST'])
def configure_ssh_all():
    """Endpoint para configurar SSH en todos los routers."""
    print("[INFO] Conexión entrante a /ssh: Iniciando configuración de SSH en los routers...")
    try:
        data = request.get_json()
        new_username = data.get('username', 'sshuser')
        new_password = data.get('password', 'sshpass')

        # Lista de IPs de los routers
        router_ips = [
            '192.168.122.201', # R201
            '192.168.122.202', # R202
            '192.168.122.203', # R203
            '192.168.122.204'  # R204
        ]

        results = []
        for ip in router_ips:
            router = RouterConfig(ip)

            result = router.configure_ssh(new_username, new_password)
            results.append({
                "router": ip,
                "status": result["status"],
                "message": result["message"]
            })

        print("[INFO] Configuración SSH completada en todos los routers")
        return jsonify({
            "status": "completed",
            "results": results
        })

    except Exception as e:
        error_message = str(e)
        print(f"[ERROR] {error_message}")
        return jsonify({
            "status": "error",
            "message": error_message
        }), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Imagen 6. Script de Python para la API-REST

Con la configuración del script de Python, se ha logrado configurar correctamente el protocolo SSH

```
[INFO] Ejecutando comando en 192.168.122.204: line vty 0 4
[INFO] Ejecutando comando en 192.168.122.204: transport input ssh telnet
[INFO] Ejecutando comando en 192.168.122.204: login local
[INFO] Ejecutando comando en 192.168.122.204: exit
[INFO] Ejecutando comando en 192.168.122.204: ip ssh version 2
[INFO] Ejecutando comando en 192.168.122.204: ip ssh authentication-retries 3
[INFO] Ejecutando comando en 192.168.122.204: ip ssh time-out 60
[INFO] Ejecutando comando en 192.168.122.204: end
[SUCCESS] SSH configurado exitosamente en 192.168.122.204
[INFO] Configuración SSH completada en todos los routers
192.168.122.50 - - [14/Nov/2024 20:39:43] "POST /ssh HTTP/1.1" 200 -
```

Imagen 7. Mensaje de éxito en la API-REST

Y con esto ya existe SSH en todos los routers

```
!
gatekeeper
shutdown
!
!
line con 0
exec-timeout 0 0
privilege level 15
logging synchronous
stopbits 1
line aux 0
exec-timeout 0 0
privilege level 15
logging synchronous
stopbits 1
line vty 0 4
login local
transport input telnet ssh
!
!
end
```

Imagen 8. SSH configurado en todos los routers